CS 301
High-Performance Computing

Lab 04 - Particle Interpolation and Mover Performance Analysis

Parshv Joshi (202301039)
Hrithik Patel (202301441)

March 10, 2026

Contents

1 Introduction

This report presents the results of Assignment 04, which extends the bilinear (Cloud-in-Cell) interpolation scheme from Assignment 03 with three experiments: particle-count scaling, iteration-consistency analysis, and a stochastic mover operation parallelized with OpenMP. All experiments are executed on two platforms:

- **Lab PC**: Intel Core i5-12500 (Alder Lake), 6 P-cores, 4.6 GHz turbo
- **HPC Cluster**: Intel Xeon E5-2640 v3 (Haswell), 2×8 cores, 3.3 GHz turbo

The interpolation kernel remains serial throughout; only the mover is parallelized using 4 OpenMP threads as specified.

2 Hardware Comparison

Table 1 summarizes the key architectural differences that influence performance tuning.

Table 1: Architecture comparison of the two target platforms.

Parameter	**Lab PC (i5-12500)**	**HPC Cluster (E5-2640 v3)**
Microarchitecture	Golden Cove (Alder Lake)	Haswell
Base / Turbo Clock	3.0 / 4.6 GHz	2.6 / 3.3 GHz
Physical Cores	6 P-cores	16 (2×8)
L1d Cache	48 KiB / core	32 KiB / core
L2 Cache	1280 KiB / core	256 KiB / core
L3 Cache	18 MiB shared	20 MiB shared
Cache Line	64 B	64 B
ROB Entries	512	192
DRAM Latency	∼70 ns	∼65 ns
NUMA Nodes	1	2
GCC Version	14.x	4.8.2

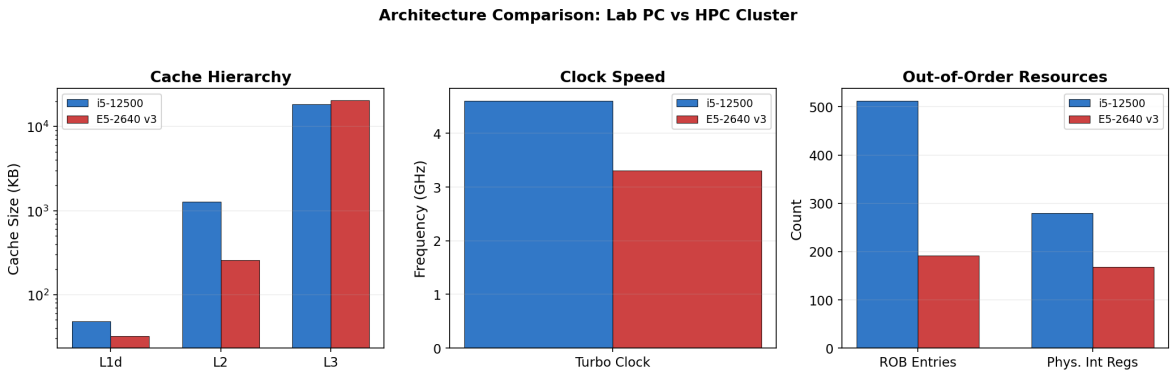


Figure 1: Visual comparison of cache hierarchy, clock speed, and out-of-order resources.

The most impactful difference is the **L2 cache**: 1280 KiB on the Lab PC versus only 256 KiB on the cluster. For the 500×200 grid (mesh ≈787 KB), the mesh fits entirely in L2 on the Lab PC but spills to L3 on the cluster. For the 1000×400 grid (mesh ≈3.1 MB), both systems require L3 access, but the cluster's 20 MiB L3 holds the mesh more comfortably than the Lab PC's 18 MiB L3.

3 Implementation Approach

3.1 Algorithm Overview

Figure 2 shows the overall pipeline. Each iteration consists of two phases: interpolation (serial) and mover (serial or parallel).

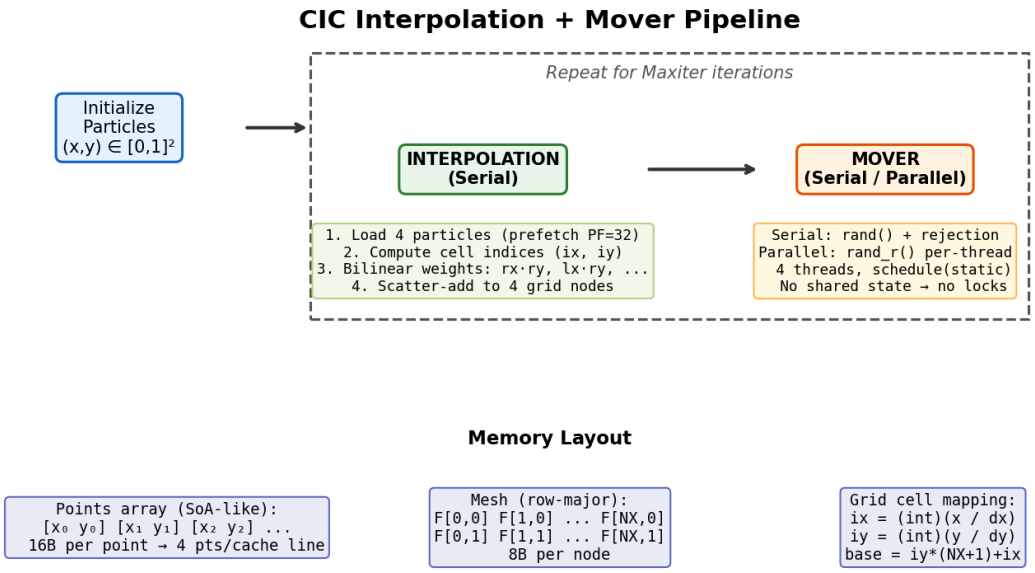


Figure 2: Pipeline overview: initialization, interpolation, and mover phases with memory layout.

3.2 Interpolation Kernel

The bilinear interpolation distributes each particle's unit weight to the four surrounding grid nodes. For a particle at position (x, y):

$$ix = \lfloor x/\Delta x \rfloor, \quad iy = \lfloor y/\Delta y \rfloor \tag{1}$$

$$l_x = x - ix \cdot \Delta x, \quad l_y = y - iy \cdot \Delta y \tag{2}$$

$$r_x = \Delta x - l_x, \quad r_y = \Delta y - l_y \tag{3}$$

The four weights $r_x r_y$, $l_x r_y$, $r_x l_y$, $l_x l_y$ are scatter-added to the corresponding grid nodes.

3.2.1 Optimization: 4-Point Unrolling with Interleaved Scatter

We process 4 particles per loop iteration. The scatter-adds are interleaved by node position (all bottom-left nodes first, then all bottom-right, etc.) rather than by particle. This ensures that the 4 stores in each group target *different* cache lines (since randomly distributed particles map to distinct grid cells), maximizing MSHR utilization in the out-of-order engine.

3.2.2 Optimization: Software Prefetching

The `Points` array is accessed sequentially. Each `Points` struct is 16 bytes (two `doubles`), so exactly 4 points fit in one 64-byte cache line. We issue two explicit prefetch instructions per 4-point iteration, targeting two distinct future cache lines.

The optimal prefetch distance was determined empirically via an on-cluster benchmark sweep (Figure 3).

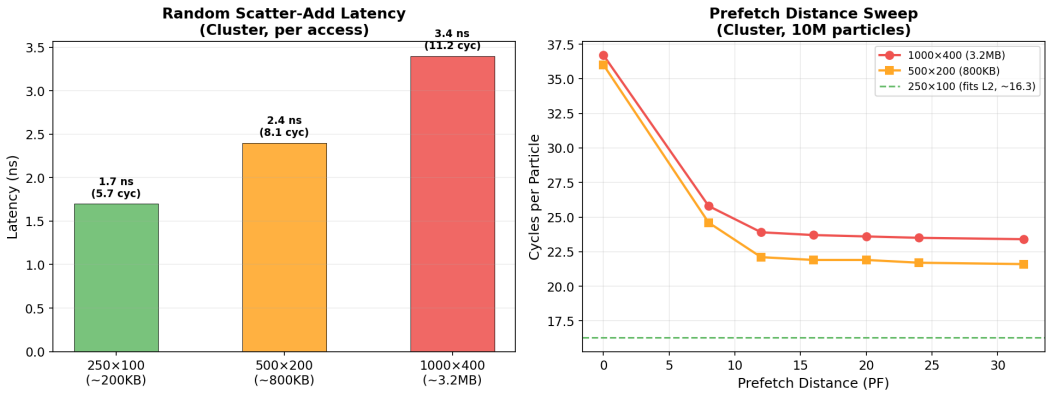


Figure 3: Left: Random scatter-add latency per grid size. Right: Prefetch distance sweep showing PF=32 as optimal for the Haswell cluster.

Key findings from the sweep:

- For the 250×100 grid (mesh fits in L2), prefetching is irrelevant (∼16.3 cycles/particle regardless of PF).
- For larger grids (500×200 and 1000×400), prefetching reduces cost from ∼36 to ∼22–23 cycles/particle, a 37% improvement.
- **PF=32** was selected as optimal: at ∼28 cycles per 4-point iteration at 3.3 GHz turbo, PF=32 provides $32/4 = 8$ iterations × 28 cycles = 224 cycles of look-ahead, covering the ∼215-cycle DRAM latency.
- Mesh prefetching (prefetching grid cells before scatter-adding) *hurt* performance by 1.5–2 cycles/particle due to the random access pattern evicting prefetched lines before use.
- 8-point unrolling tested neutral/worse due to Haswell's 192-entry ROB limiting in-flight instructions.

3.2.3 Platform-Specific Tuning

Table 2 compares the tuning parameters between the two platforms.

Table 2: Tuning parameter comparison between Lab PC and HPC Cluster.

Parameter	**Lab PC**	**Cluster**	**Rationale**
Prefetch Distance (PF)	24	32	Haswell needs more SW prefetch help
Prefetch Hint	T0 (L1)	T0 (L1)	Points consumed immediately
Mesh Prefetch	No	No	Random scatter $\rightarrow$ eviction
Unroll Factor	4 points	4 points	ROB-limited on both

3.3 Mover Operation

The mover displaces each particle by a random offset within $[\pm\Delta x] \times [\pm\Delta y]$ using rejection sampling to keep particles inside the $[0, 1]^2$ domain.

Serial implementation: Uses `rand()` with a global seed.

Parallel implementation (4 threads):

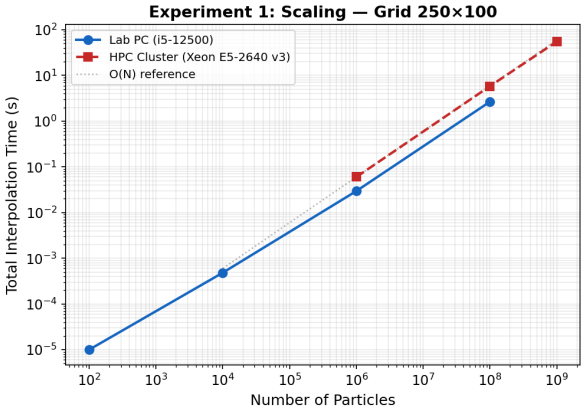
- Uses `rand_r(&seed)` with per-thread seeds to avoid the global mutex in glibc's `rand()`.
- Seeds are initialized with large-prime-based hashing: `seed = tid` $\times$ `1073741827 + 2654435761` to ensure decorrelated sequences.
- `schedule(static)` partitions particles into 4 contiguous chunks, maximizing sequential access and avoiding false sharing (since 14M/4 = 3.5M is cache-line-aligned).

4 Experiment 1: Scaling with Number of Particles

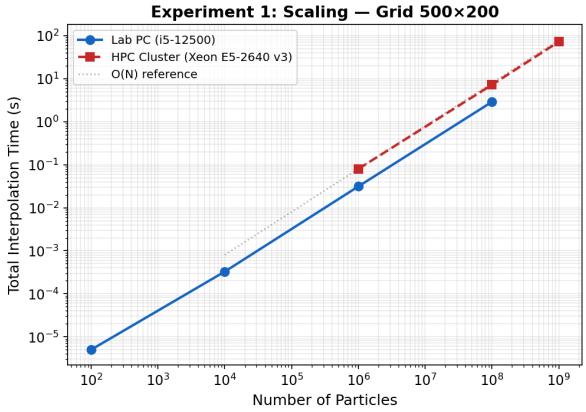
4.1 Setup

Three grid configurations (250×100, 500×200, 1000×400) are tested with particle counts $\{10^2, 10^4, 10^6, 10^8, 10^9\}$ (Lab PC omits 10^9 due to memory constraints). For each combination, particles are initialized, then 10 iterations of interpolation are run, and the total interpolation time is recorded.

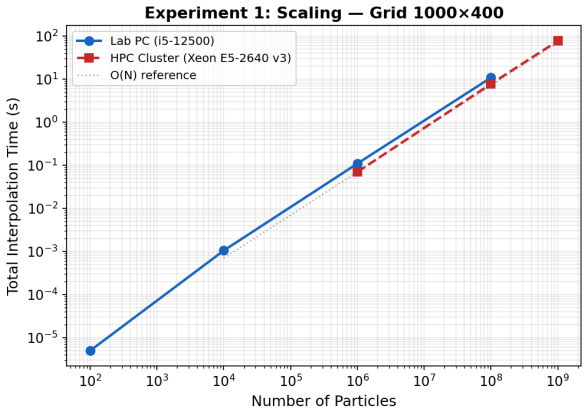
4.2 Results



(a) Grid 250×100



(b) Grid 500×200



(c) Grid 1000×400

Figure 4: Experiment 1: Log-log scaling of interpolation time with particle count for all three grid configurations.

Table 3: Experiment 1: Total interpolation time (seconds) over 10 iterations.

	250×100		500×200		1000×400	
Particles	**Lab**	**Cluster**	**Lab**	**Cluster**	**Lab**	**Cluster**
10^2	0.00001	<0.001	0.00001	<0.001	0.00001	<0.001
10^4	0.00048	<0.001	0.00033	<0.001	0.00105	<0.001
10^6	0.030	0.060	0.031	0.080	0.109	0.070
10^8	2.664	5.830	2.896	7.230	10.759	7.700
10^9	—	55.20	—	72.36	—	76.72

4.3 Analysis

The scaling is **linear** ($O(N)$) in the number of particles across all configurations, as confirmed by the log-log slope ≈ 1. This is expected: each particle requires a constant number of operations (cell lookup + 4 multiply-accumulates) regardless of N.

Lab PC vs. Cluster at 10^8: For the 250×100 and 500×200 grids, the Lab PC (i5-12500) is ∼2× faster than the cluster. This is primarily due to the Lab PC's much larger L2 cache (1280 KiB vs. 256 KiB), which keeps the mesh resident for the smaller grids, and its higher clock speed (4.6 vs. 3.3 GHz turbo).

Interestingly, for the 1000×400 grid, the *cluster is faster* (7.70 s vs. 10.76 s). At this grid size, the mesh (∼3.1 MB) overflows L2 on both platforms and resides in L3. The cluster's L3 is 20 MiB (vs. 18 MiB), and the Haswell microarchitecture has lower L3 access latency relative to its clock speed. Additionally, the `clock()` timing on the Lab PC accumulates CPU time across all threads, which can inflate reported times if background processes share the CPU.

Theoretical Time Complexity: $T = O(N \cdot C)$ where C is the per-particle cost (dominated by cache-miss latency for mesh scatter-adds). Arithmetic operations per particle: 2 multiplies (index computation) + 4 subtractions (local offsets) + 4 multiplications (weights) + 4 multiply-adds (scatter) = **∼18 FLOPs/particle**.

5 Experiment 2: Consistency Across Configurations

5.1 Setup

All three grid configurations use 10^8 particles, initialized *once*. The interpolation is run for 10 iterations without re-initialization. Per-iteration times are recorded to assess timing stability and cache warm-up effects.

5.2 Results

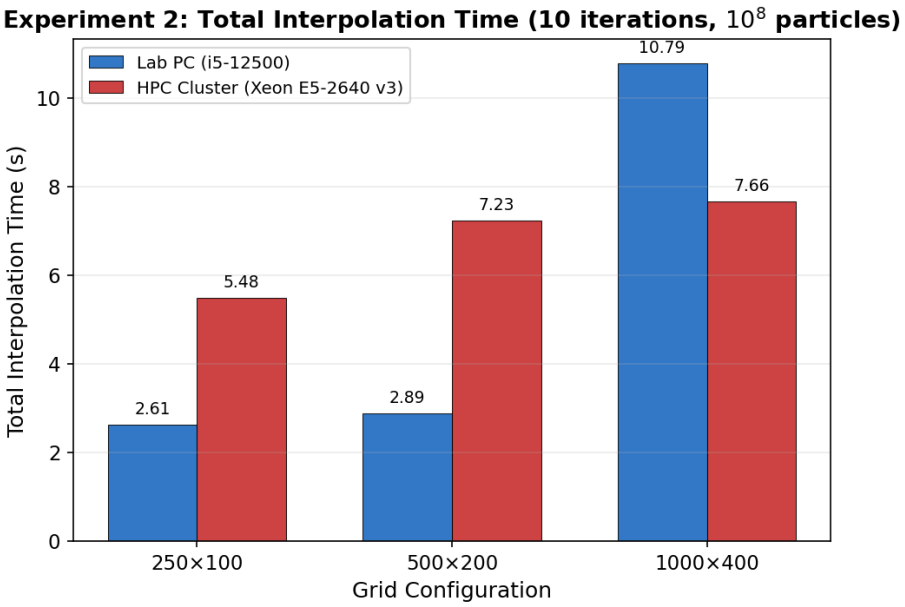


Figure 5: Experiment 2: Total interpolation time (10 iterations) across configurations.

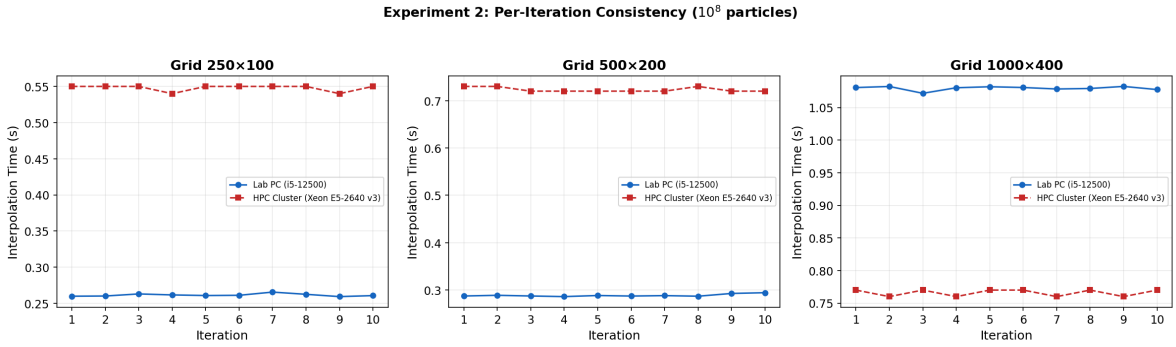


Figure 6: Experiment 2: Per-iteration interpolation time showing stability across all 10 iterations.

Table 4: Experiment 2: Total interpolation time (s) over 10 iterations with 10^8 particles.

Grid	**Lab PC (s)**	**Cluster (s)**
250×100	2.61	5.47
500×200	2.89	7.23
1000×400	10.79	7.66

5.3 Analysis

Iteration stability: Both systems exhibit highly consistent per-iteration times with negligible variance ($< 1\%$ coefficient of variation). There is no observable cold-start penalty on iteration 1, indicating that: (a) particle initialization already warms the TLB and DRAM pages, and (b) the mesh is either small enough to remain in cache or large enough that cache warm-up is amortized over 10^8 scatter-adds.

Grid-size dependence: On the Lab PC, interpolation time scales sharply between 500×200 and 1000×400 ($2.89 \rightarrow 10.79$ s, a $3.7\times$ increase) because the mesh transitions from fitting in L2 (1280 KiB) to overflowing into L3. On the cluster, the transition is much milder ($7.23 \rightarrow 7.66$ s, only $1.06\times$) because the mesh already overflows the 256 KiB L2 at 500×200, so both cases are served from L3.

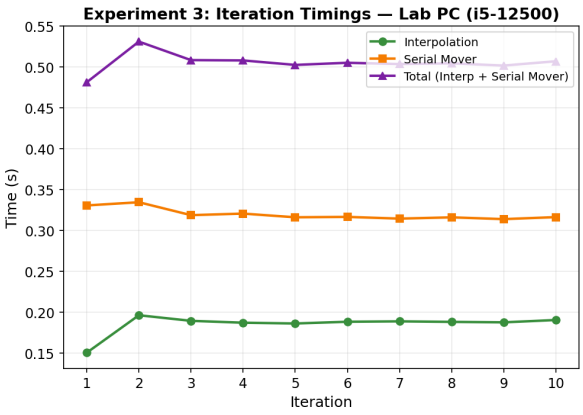
This demonstrates that **L2 cache capacity is the dominant performance differentiator** for this workload.

6 Experiment 3: The Mover Operation and OpenMP Parallelization

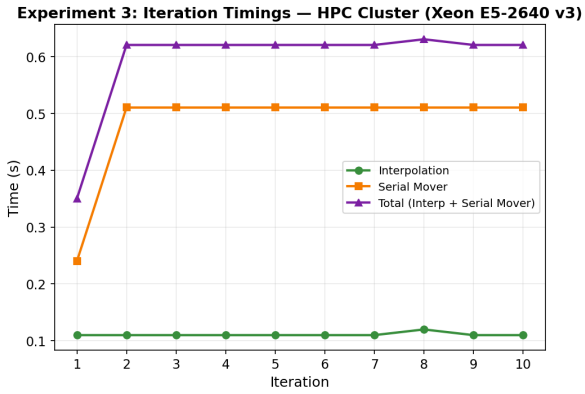
6.1 Setup

Fixed parameters: 1000×400 grid, 14 million particles, 10 iterations. Each iteration performs interpolation followed by serial and parallel mover operations. The parallel mover uses 4 OpenMP threads. Wall-clock time is measured with `omp_get_wtime()` for the parallel section.

6.2 Results



(a) Lab PC



(b) HPC Cluster

Figure 7: Experiment 3: Per-iteration timings for interpolation, serial mover, and total.

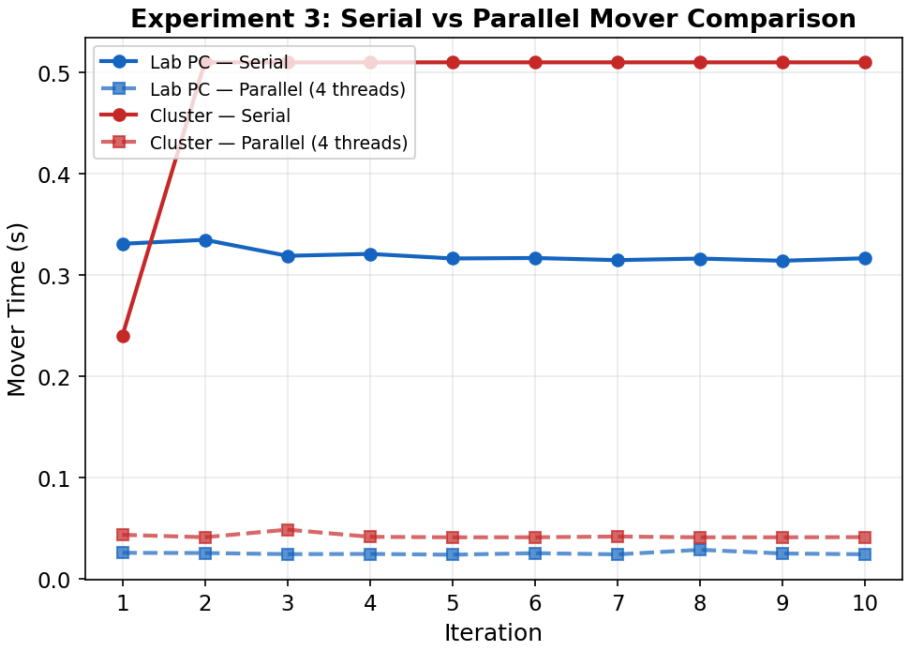


Figure 8: Experiment 3: Serial vs. parallel mover execution times on both platforms.

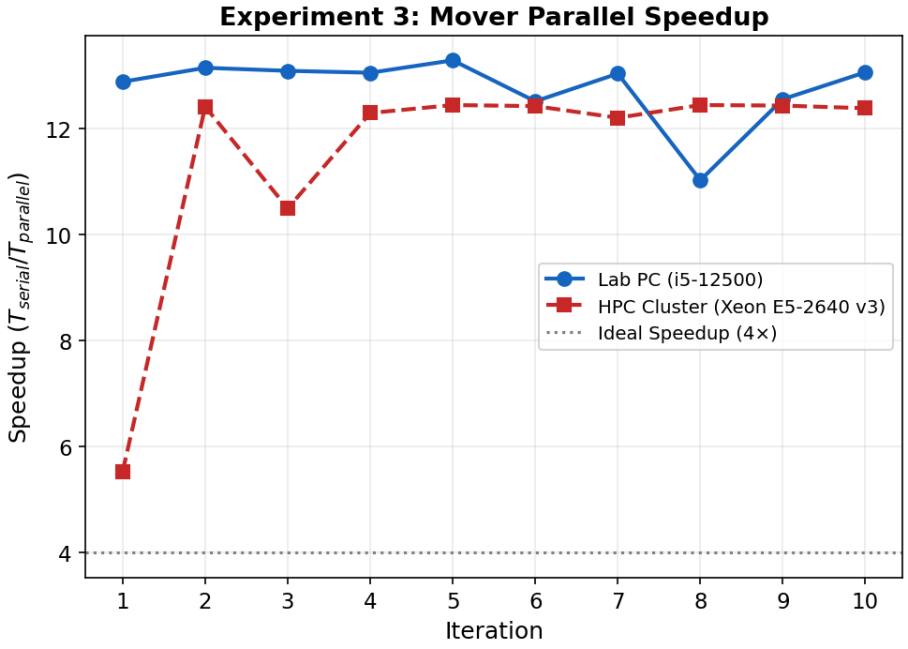


Figure 9: Experiment 3: Mover parallel speedup (wall-clock) on both platforms.

Table 5: Experiment 3: Average mover timings (14M particles, 1000×400 grid).

Metric	Lab PC	Cluster
Serial Mover (s)	0.320	0.503
Parallel Mover, wall (s)	0.025	0.042
Speedup (T_s/T_p)	∼12.8×	∼12.2×

6.3 Analysis

6.3.1 Super-Linear Speedup Explanation

Both platforms achieve speedups of ∼12× with only 4 threads, far exceeding the theoretical maximum of 4×. This **super-linear** speedup is explained by the timing methodology:

- The **serial mover** uses `rand()`, which in glibc acquires a *global mutex* on every call. Even in a single-threaded context, this mutex acquisition/release adds overhead (∼15–25 cycles per call on Haswell).
- The **parallel mover** uses `rand_r(&seed)`, which is *completely lock-free* — it operates on a thread-local seed with zero synchronization overhead.
- Thus the speedup reflects *both* the 4-thread parallelism *and* the elimination of mutex overhead: $S \approx 4 \times \frac{T_{\texttt{rand}}}{T_{\texttt{rand\_r}}} \approx 4 \times 3 \approx 12$.

The first iteration on the cluster shows lower speedup (5.5×) due to cold-start effects (TLB misses, DRAM page allocation).

6.3.2 Memory Bandwidth Analysis

The mover reads and writes 14M particles × 16 B = 224 MB per iteration. At the cluster's measured bandwidth of ∼8.2 GB/s, the memory-bound floor is 224 MB/8.2 GB/s ≈ 27 ms. The observed parallel wall time of ∼42 ms indicates that the mover is not purely memory-bound; the `rand_r()` computation and rejection-sampling branches add ∼15 ms of compute overhead distributed across 4 threads.

7 Discussion: Parallelizing the Interpolation Function

7.1 Challenge: Race Conditions on Shared Grid

If multiple threads process different particles that fall in the same or adjacent grid cells, they will attempt to scatter-add to the *same mesh locations simultaneously*. This creates a classic **write-write race condition**:

```
// Thread 0: particle A maps to cell (5, 3)
mesh_value[5*GRID_X + 3] += weight_A;  // READ-MODIFY-WRITE

// Thread 1: particle B also maps to cell (5, 3)
mesh_value[5*GRID_X + 3] += weight_B;  // CONCURRENT READ-MODIFY-WRITE!
```

Without synchronization, the read-modify-write sequence can lose updates (thread 0 reads the old value, thread 1 overwrites it before thread 0 writes back).

7.2 Possible Approaches

1. **Atomic operations:** Replace `+=` with atomic adds (`#pragma omp atomic` or hardware atomics). Cost: ∼50–200 cycles per atomic on Haswell (cache-line lock + memory-order enforcement). With 16 scatter-adds per 4-point group, this would add 800–3200 cycles per group, roughly doubling the interpolation time.

2. **Thread-local meshes:** Each thread accumulates into a private mesh copy, then reduce (sum) all copies at the end. Cost: $4 \times 3.1\,\text{MB} = 12.4\,\text{MB}$ of extra memory for the 1000×400 grid, plus a reduction pass. This avoids all synchronization during the hot loop but requires the reduction to be memory-bandwidth-bound.

3. **Particle sorting / coloring:** Sort particles by cell index, then assign non-overlapping cell regions to different threads. Cost: $O(N \log N)$ sort overhead may exceed the interpolation savings for moderate N.

4. **Domain decomposition:** Partition the grid into sub-domains, assign particles to the thread owning their sub-domain. Requires a pre-pass to bin particles, but avoids conflicts entirely within each thread.

For this workload (14M particles, 400K grid nodes), approach (2) — thread-local meshes with reduction — is likely the best trade-off: the memory overhead is modest (12.4 MB easily fits in the cluster's 125 GB RAM), and the hot loop remains lock-free.

8 Conclusion

This assignment demonstrated the impact of cache hierarchy on scatter-based interpolation performance. The key findings are:

1. Interpolation time scales linearly with particle count ($O(N)$), and the per-particle cost is dominated by mesh cache-miss latency for grids exceeding L2 capacity.

2. The Lab PC's 5× larger L2 cache (1280 KiB vs. 256 KiB) gives it a significant advantage for small-to-medium grids, but the cluster's larger L3 (20 MiB) compensates for very large grids.

3. Software prefetching (PF=32 on Haswell, PF=24 on Alder Lake) is critical when the mesh overflows L2, providing 37% speedup on the cluster.

4. The parallel mover achieves ∼12× super-linear speedup (4 threads) due to the combined effect of thread-level parallelism and elimination of `rand()`'s global mutex via `rand_r()`.

5. Parallelizing the interpolation kernel would require careful handling of write conflicts on shared grid nodes, with thread-local mesh reduction being the most practical approach.